

Real Failures & Debugging

Use observed incidents • Explain your reasoning • Avoid invented production claims

SAY IT IN THE INTERVIEW

“One difficult issue in my local project was an outbox publisher that appeared stuck. I traced the blocking path to Avro schema registration over HTTP, before Kafka returned the send future. I adjusted the timeout layers, added correlated logs and verified the business flow after recovery.”

01 STORY 1 — THE OUTBOX THAT LOOKED HUNG

- **Situation:** an order existed but publication was delayed. I followed `eventId/orderId` through `OutboxPublisher` → `OrderEventProducer` → `KafkaAvroSerializer` → Registry registration.
- **Evidence:** the documented local run observed one Registry request taking about **26.9 seconds**. A temporary 10-second HTTP limit was too aggressive for that run.
- **Action:** timeout settings and interruption handling were hardened (Chapter 03). **Result:** the recorded verification reached `PUBLISHED` and `INVENTORY_RESERVED`, with no pending Order outbox rows from that run.

02 STORY 2 — CONTAINER STARTED, DEPLOYMENT FAILED

- **Situation:** starting a Docker container did not establish readiness. The earlier curl retry did not necessarily retry the complete health predicate.
- **Action:** retry curl success **and** the UP-content check together. **Result:** the Jenkinsfile now contains the full-condition loop; Chapter 13 explains its exact boundary.
- **Lesson:** distinguish process creation from application readiness. Do not invent an outage duration or a performance improvement percentage.

03 STORY 3 — LOCAL RESOURCE PRESSURE

- **Situation:** the documented 8 GB development laptop showed pressure while running the full stack, alongside slow startup and intermittent failures.
- **Recovery:** reduce competing load, recover brokers, then start Registry after broker health. Compose includes broker-health dependencies.
- **Lesson:** local resource pressure was an observed contributor, not a proven explanation for every broker timeout or a Kafka architecture defect.

04 A SIMPLE ANSWER STRUCTURE

- Use **symptom** → **evidence** → **change** → **verification** → **remaining limit**. State which part you personally implemented or debugged.
- Say **“In my personal hands-on project...”** for these stories. Company project, scale, production traffic and team ownership require your actual work history.

YAAD RAKHO: symptom → evidence → action → verified result → lesson.